

# QA Team Setup

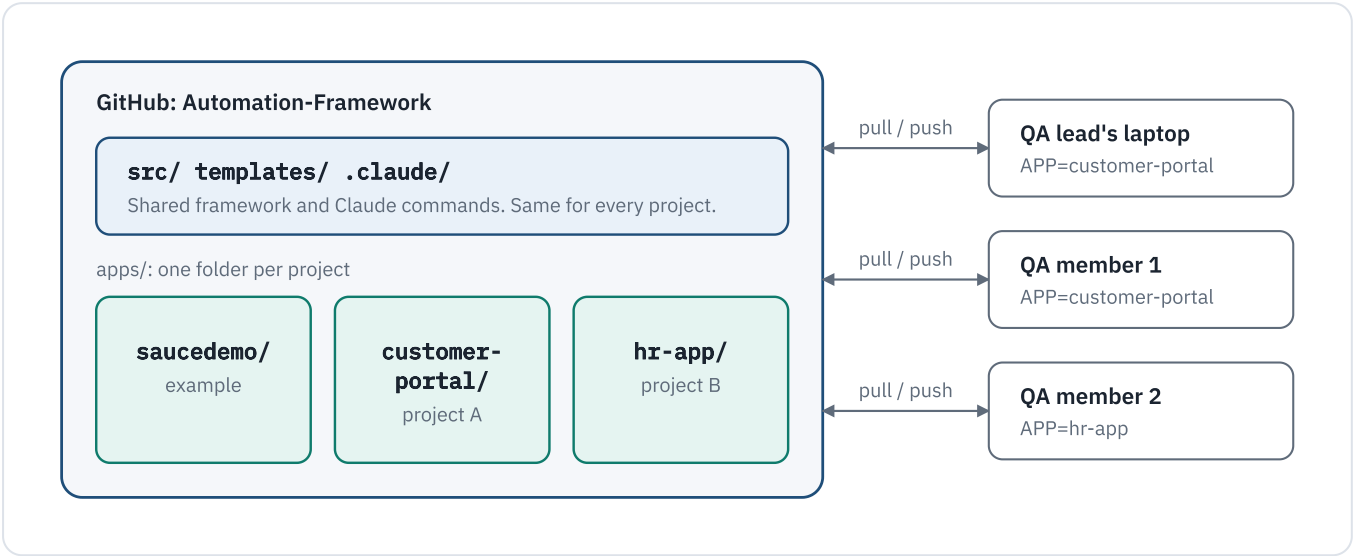
Everything a new QA team member needs to get the framework running on their machine and start working. Everyone uses the same GitHub repository; there is nothing else to install or publish.

|                                           |                                    |                                                     |
|-------------------------------------------|------------------------------------|-----------------------------------------------------|
| <b>TIME TO SET UP</b><br>About 30 minutes | <b>COST</b><br>Free, except Claude | <b>PROJECTS</b><br>All in one repo, one folder each |
|-------------------------------------------|------------------------------------|-----------------------------------------------------|

## THE IDEA

### How the team shares the framework

One repository holds the shared framework and every project. Each person clones it once; `git pull` brings everyone the latest framework fixes and tests.



Each person picks the project they work on with `APP=` in their own root `.env`. Passwords never go through GitHub.

## MONEY

### What it costs

Only Claude is paid. Without it the team can still run, edit and report on tests; only the AI commands are missing.

| ITEM                                           | COST | NOTES                                                        |
|------------------------------------------------|------|--------------------------------------------------------------|
| Framework, Playwright, Allure, Appium, Node.js | Free | Open source                                                  |
| VS Code, Git                                   | Free |                                                              |
| GitHub private repository                      | Free | Each member needs a GitHub account with access to the repo   |
| Claude Code (the /qa-* commands)               | Paid | A Claude subscription per QA member who uses the AI commands |

#### STEP 1 · ONCE PER MACHINE

## Install the tools

Install these four, plus access to the repository. Check each with the command shown.

### Node.js 20 or newer

Runs the framework. [nodejs.org](https://nodejs.org), LTS version.

Check: `node -v`

### Git

Gets and shares the code. [git-scm.com](https://git-scm.com).

Check: `git --version`

### VS Code

The editor everyone uses.  
[code.visualstudio.com](https://code.visualstudio.com).

Check: it opens

### Claude Code extension

In VS Code: Extensions → search "Claude Code" → Install, then sign in.

Check: the Claude panel opens

### Ask the QA lead for

- Access to [github.com/jibints-smb/Automation-Framework](https://github.com/jibints-smb/Automation-Framework)
- The project's `.env` file (accounts and settings), through the password manager

- Which project ( APP ) you'll work on

## STEP 2 · ONCE PER MACHINE

# First-time setup

Run these in a terminal (in VS Code: Terminal → New Terminal). Commands are for Windows; on macOS / Linux use `cp` instead of `copy`.

### 1 Get the framework

```
git clone https://github.com/jibints-smb/Automation-Framework.git
cd Automation-Framework
```

### 2 Install the packages and browsers

```
npm install --include=dev          # --include=dev matters on
machines with NODE_ENV=production
npm run setup                      # downloads Chromium + WebKit for
Playwright
```

### 3 Create your root settings file and pick the project

```
copy .env.example .env
```

Open the new `.env` in the root folder and set the project you work on:

```
APP=customer-portal              # folder name under apps/
TEST_ENV=qa
```

This root file holds no passwords, only which project and environment you test.

### 4 Add the project's settings file

Save the `.env` the QA lead gave you as `apps/customer-portal/.env`. It holds the QA test accounts, so it is never committed (Git ignores it).

### 5 Open the folder in VS Code and turn on the browser tool

File → Open Folder → `Automation-Framework` (the root folder, so Claude finds the `/qa-*` commands). Open the Claude panel; when Claude asks to use the **playwright** server,

approve it. Type `/mcp` to see it listed.

ONLY IF /MCP DOESN'T LIST PLAYWRIGHT

```
claude mcp add playwright --scope user -- cmd /c npx -y
@playwright/mcp@latest --isolated --output-dir=.playwright-mcp
```

Then reload VS Code and start a new Claude conversation. On macOS / Linux drop `cmd /c .`

### STEP 3

## Check it works

Three commands. If all three pass, you're ready.

---

```
npm run auth          # logs in every role of your project; all must
pass
npm run test:smoke     # the quick suite
npm run report         # opens the Allure report in your browser
```

### No project yet?

Use the example project to check your setup. Its example settings file already holds the public demo accounts:

```
copy apps\saucedemo\.env.example apps\saucedemo\.env
# set APP=saucedemo in the root .env, then:
npm run auth
npm run test:web
```

### EVERY DAY

## Daily work

Slash commands go in the Claude panel; `npm run` commands in the terminal.

---

| I WANT TO...                          | DO THIS                                                                                                            |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Start the day with the latest code    | <code>git pull</code>                                                                                              |
| Make test cases from a story          | <code>/qa-testcases apps/&lt;app&gt;/requirements/web/&lt;file&gt;.md</code>                                       |
| Automate reviewed test cases          | <code>/qa-automate apps/&lt;app&gt;/test-cases/web/&lt;file&gt;.testcases.md</code>                                |
| Fix failing tests                     | <code>/qa-fix</code> (or <code>/qa-fix @smoke</code> , or a spec path)                                             |
| Update tests after a story changed    | <code>npm run req:status</code> → <code>npm run req:diff -- &lt;md&gt;</code> → <code>/qa-update &lt;md&gt;</code> |
| Run one spec while working            | <code>npx playwright test &lt;file&gt; --project=web-chrome</code>                                                 |
| Run a suite                           | <code>npm run test:smoke</code> · <code>npm run test:regression</code>                                             |
| See the report                        | <code>npm run report</code> · all saved runs: <code>npm run reports</code>                                         |
| See which test cases aren't automated | <code>npm run coverage:tc</code>                                                                                   |

The sprint routine (planning, build day, sign-off) is in the [Sprint QA Process](#) page.

## EVERY DAY

# Sharing your work

Work on a branch and open a pull request, so someone reviews before it reaches `main`.

```
git pull                                # always start from the
latest main
git switch -c prj-101-login             # a branch per story or task
... work: requirements, test cases, tests ...
git add apps/customer-portal
git commit -m "PRJ-101 login: test cases and automation"
```

```
git push -u origin prj-101-login  
on GitHub
```

```
# then open a pull request
```

## Commit

- ✓ Requirements, test cases, models, pages, data, specs
- ✓ `requirements/.baseline/` (written by the commands)
- ✓ `sprints/` files and developer MDs
- ✓ Approved screenshots in `screenshots/`

## Never commit

- `.env` files (Git already ignores them)
- `.auth/` saved logins, or any zip of them
- Reports and test results folders
- Changes to `src/` without the lead's review

## PEOPLE

# QA lead vs team members

### QA lead / framework owner

- Gives repo access and shares `.env` files through the password manager
- Creates new projects ( `npm run new:app` ) and the login setup
- Reviews pull requests; merges to `main`
- Owns changes to `src/` , templates and `.claude/` commands
- Records sign-off: `req:baseline -- stage signed-off`

### QA team member

- Writes requirement MDs and reviews test cases
- Automates with `/qa-automate` , triages with `/qa-fix`
- Raises bugs in Jira, adds `knownBug` to tests
- Works on branches, opens pull requests
- Suggests framework changes to the lead instead of editing `src/`

## OCCASIONALLY

# Adding a new project

One person (usually the lead) does it once; everyone else just pulls.

1

## Create and configure it

```
npm run new:app -- customer-portal --platforms web
```

Fill in `apps/customer-portal/app.config.ts` and `.env`, automate the login, run `npm run auth`. Details: [Starting a new web app](#).

2

## Share it

Commit and push (without the `.env`). Send the `.env` to the team through the password manager.

3

## Everyone else

`git pull`, save the `.env` as `apps/customer-portal/.env`, set `APP=customer-portal`, run `npm run auth`.

## HELP

# Common problems

### "Cannot find module '@playwright/test'" or other missing packages

The dev packages were skipped. Run `npm install --include=dev`.

### "Executable doesn't exist" / browser not found

Run `npm run setup` to download the browsers.

### Tests run against the wrong project

Check `APP=` in the root `.env`, or run with `APP=customer-portal` in front of the command.

### `npm run auth` fails

Check the accounts in `apps/<app>/.env` (variable names must match `auth.roles` in `app.config.ts`), and that the QA site is up. Try logging in by hand with the same account.

### The `/qa-*` commands don't appear in Claude

VS Code must have the `root Automation-Framework` folder open, not a subfolder. Reload the window after `git pull`.

### Claude can't open the browser

Type `/mcp` in the Claude panel. If playwright isn't listed or not approved, use the command in step 2.5, reload VS Code and start a new conversation.

### `git pull` says my changes would be overwritten

Commit your work on a branch first (see Sharing your work), then pull.

NEXT

## What to read next

---

### Getting Started Guide

How Playwright + Claude automation works, the framework's architecture, a full walkthrough.

`docs/Getting-Started-Guide.pdf`

### Sprint QA Process

Planning, build day, testing and sign-off for each sprint.

`docs/sprint-qa-process.html`

### QA Automation Guide

What a module MD must contain, test IDs, severity, re-testing.

`docs/QA-Automation-Guide.pdf`

---

Questions or setup problems: ask the QA lead. Framework rules Claude follows: `CLAUDE.md`.